

OPD-VERTEX -- SQL + NoSQL Schema & Data Flow

Group 3

Date: March 2026

Part 1: SQL Database (MySQL)

Table 1: staff

Stores all registered accounts (doctors and admins). Admins leave medical fields NULL.

Field	Type	Constraints	Description
staff_id	INT	PK, AUTO_INCREMENT	Unique staff identifier
first_name	VARCHAR(100)	NOT NULL	First name
last_name	VARCHAR(100)	NOT NULL	Last name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
specialization	VARCHAR(150)	NULL	Medical specialization (NULL for admins)
license_number	VARCHAR(100)	NULL, UNIQUE	Medical license (NULL for admins)
phone	VARCHAR(20)	NULL	Contact phone
role	ENUM('doctor','admin')	NOT NULL, DEFAULT 'doctor'	RBAC role
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Created
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Last updated

Indexes: email (UNIQUE), license_number (UNIQUE)

Table 2: patients

Stores patient demographic and contact data.

Field	Type	Constraints	Description
patient_id	INT	PK, AUTO_INCREMENT	Unique patient identifier
first_name	VARCHAR(100)	NOT NULL	First name
last_name	VARCHAR(100)	NOT NULL	Last name
date_of_birth	DATE	NOT NULL	Date of birth
gender	ENUM('M','F','Other')	NULL	Gender
email	VARCHAR(255)	NOT NULL	Email for prescriptions
phone	VARCHAR(20)	NULL	Contact phone
address	TEXT	NULL	Home address
emergency_contact	VARCHAR(255)	NULL	Emergency contact name + phone
blood_type	VARCHAR(5)	NULL	Blood type (e.g. A+, O-)
allergies	TEXT	NULL	Known allergies
medical_history	TEXT	NULL	Brief medical history
insurance_id	VARCHAR(100)	NULL	Insurance/health card number

Field	Type	Constraints	Description
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Created
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Last updated

Indexes: email, date_of_birth

Table 3: consultations

Structured metadata per consultation session. Transcript text and AI outputs live in NoSQL.

Field	Type	Constraints	Description
consultation_id	INT	PK, AUTO_INCREMENT	Unique consultation identifier
doctor_id	INT	FK -> staff.staff_id, ON UPDATE CASCADE,	Conducting doctor
patient_id	INT	FK -> patients.patient_id, ON UPDATE CASCADE, ON DELETE RESTRICT	Patient in session
status	ENUM('recording','transcribing','processing')	DEFAULT 'recording'	Workflow state
started_at	DATETIME	NOT NULL	Recording start
ended_at	DATETIME	NULL, CHECK (ended_at IS NULL OR ended_at >= started_at)	Recording end
approved_at	DATETIME	NULL, CHECK (approved_at IS NULL OR approved_at >= started_at)	Doctor approval time
transcript_doc_id	VARCHAR(64)	NULL	MongoDB ObjectId for raw transcript
notes_doc_id	VARCHAR(64)	NULL	MongoDB ObjectId for AI-generated notes
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Created
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last updated

Indexes: doctor_id, patient_id, status

Table 4: prescriptions

Finalized prescription data + PDF delivery status. Data lands here only AFTER doctor approval.

Field	Type	Constraints	Description
prescription_id	INT	PK, AUTO_INCREMENT	Unique prescription identifier
consultation_id	INT	FK -> consultations	Parent consultation
doctor_id	INT	FK -> staff.staff_id	Prescribing doctor
patient_id	INT	FK -> patients.patient_id	Receiving patient
diagnosis	TEXT	NOT NULL	Final diagnosis
medications	JSON	NOT NULL	Array of medication objects
instructions	TEXT	NULL	Patient instructions
follow_up_date	DATE	NULL	Recommended follow-up
pdf_path	VARCHAR(500)	NULL	Path to generated PDF
is_approved	BOOLEAN	DEFAULT FALSE	Doctor approved
is_emailled	BOOLEAN	DEFAULT FALSE	PDF emailed to patient
emailed_at	DATETIME	NULL	Email sent timestamp

Field	Type	Constraints	Description
version	INT	DEFAULT 1	Version number
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Created
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Last updated

Indexes: consultation_id, doctor_id, patient_id

Table 5: audit_logs

Tracks all significant actions for security and compliance.

Field	Type	Constraints	Description
log_id	INT	PK, AUTO_INCREMENT	Unique log identifier
user_id	INT	NOT NULL	Acting user ID
user_role	VARCHAR(20)	NOT NULL	Role at time of action
action	VARCHAR(100)	NOT NULL	Action type (LOGIN, APPROVE, etc.)
target_table	VARCHAR(50)	NULL	Affected table
target_id	INT	NULL	Affected record ID
details	JSON	NULL	Additional context
ip_address	VARCHAR(45)	NULL	Client IP
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	When it happened

Indexes: user_id, action, timestamp

SQL Relationships

From	To	Cardinality	Meaning
staff -> consultations	staff_id -> doctor_id	1:M	One doctor -> many consultations
staff -> prescriptions	staff_id -> doctor_id	1:M	One doctor -> many prescriptions
staff -> audit_logs	staff_id -> user_id	1:M	One staff -> many log entries
patients -> consultations	patient_id -> patient_id	1:M	One patient -> many consultations
patients -> prescriptions	patient_id -> patient_id	1:M	One patient -> many prescriptions
consultations -> prescriptions	consultation_id -> consultation_id	1:M	One consultation -> multiple versions

Part 2: NoSQL Database (MongoDB)

Collection 1: email_templates

Email template for sending prescription PDFs. Stored in NoSQL so admins can update without code changes.

Key fields: _id, subject_template, body_template, placeholders[], from_email, reply_to, attachment_fields

```
{
  "_id": "prescription_delivery_v1",
  "template_name": "Prescription Delivery Email",
  "version": 1,
```

```

"subject_template": "Your Prescription from Dr. {{doctor_name}} -- {{clinic_name}}",
"body_template": "Dear {{patient_first_name}} {{patient_last_name}},\n\nThank you for visiting {{clinic_name}} on {{consultation_date}}.\n\nPlease find attached your prescription from your consultation with Dr. {{doctor_first_name}} {{doctor_last_name}} ({{doctor_specialization}}).\n\nCONSULTATION SUMMARY:\n-----\nDate: {{consultation_date}}\nTime: {{consultation_time}}\nDiagnosis: {{diagnosis}}\nFollow-up: {{follow_up_date | 'No follow-up scheduled'}}\n\nPRESCRIBED MEDICATIONS:\n{{#each medications}}\n  - {{this.name}} -- {{this.dosage}}, {{this.frequency}}, for {{this.duration}}\n  Instructions: {{this.special_instructions}}\n{{/each}}\n\nGENERAL INSTRUCTIONS:\n{{patient_instructions}}\n\n-----\n\nThis prescription has been verified and approved by Dr. {{doctor_name}}.\n\nIf you have any questions, contact the clinic at {{clinic_phone}} or reply to this email.\n\nRegards,\n{{clinic_name}}\n{{clinic_address}}\n\nDISCLAIMER: This email and the attached prescription contain confidential medical information intended only for the named patient.",
"from_email": "noreply@{{clinic_domain}}",
"reply_to": "{{doctor_email}}",
"attachment_fields": {
  "filename_template": "Prescription-{{patient_last_name}}-{{consultation_date}}.pdf",
  "mime_type": "application/pdf"
},
"placeholders": [
  "patient_first_name", "patient_last_name",
  "doctor_name", "doctor_first_name", "doctor_last_name",
  "doctor_specialization", "doctor_email",
  "clinic_name", "clinic_domain", "clinic_phone", "clinic_address",
  "consultation_date", "consultation_time",
  "diagnosis", "follow_up_date",
  "medications", "patient_instructions"
],
"created_at": "2026-02-24T00:00:00Z",
"updated_at": "2026-02-24T00:00:00Z"
}

```

Collection 2: llm_prompts

System prompts for the local LLM. Stored in DB so prompts can be updated without redeploying code.

Key fields: `_id`, `prompt_name`, `version`, `model_target`, `system_prompt`, `user_prompt_template`, `temperature`, `max_tokens`

Document 2A -- Prescription & Clinical Notes Generator

```

{
  "_id": "prescription_generation_v1",
  "prompt_name": "Prescription & Clinical Notes Generator",
  "version": 1,
  "model_target": "llama3.1-8b OR mistral-7b-v0.3",
  "system_prompt": "You are a clinical documentation assistant embedded in an outpatient clinic system. Your task is to convert a raw doctor-patient consultation transcript into structured clinical notes and a prescription draft.\n\nINPUT YOU WILL RECEIVE:\n- Patient metadata (name, age, gender, known allergies, medical history)\n- The full verbatim transcript of the consultation\n\nOUTPUT REQUIREMENTS:\nReturn a single valid JSON object with exactly these keys:\n{\n  \"patient_info\": {\n    \"name\": \"...\", \"age\": \"...\", \"gender\": \"...\" },\n  \"chief_complaint\": \"Primary reason for visit in one sentence\", \n  \"history_of_present_illness\": \"Narrative of the current issue including onset, duration, severity, aggravating/relieving factors\", \n  \"past_medical_history\": \"Relevant past conditions mentioned in the transcript\", \n  \"allergies\": \"Known allergies from patient record and any mentioned in transcript\", \n  \"vitals\": {\n    \"blood_pressure\": \"...\", \"heart_rate\": \"...\", \"temperature\": \"...\", \"respiratory_rate\": \"...\", \"spo2\": \"...\" }, \n  \"examination_findings\": \"Physical examination findings mentioned by the doctor\", \n  \"diagnosis\": \"Primary diagnosis and any differential diagnoses\", \n  \"medications\": [\n    {\n      \"name\": \"Drug name\", \n      \"dosage\": \"Amount per dose\", \n      \"frequency\": \"How often (e.g. twice daily)\", \n      \"duration\": \"Length of course (e.g. 7 days)\", \n      \"route\": \"oral / topical / IV / etc.\" \n    }, \n    {\n      \"special_instructions\": \"Take with food, avoid alcohol, etc.\" \n    } \n  ], \n  \"lab_tests_ordered\": [\"List of any tests the doctor ordered\"], \n  \"follow_up\": \"Follow-up plan and timeline\", \n  \"patient_instructions\": \"Lifestyle advice and self-care instructions given to the patient\", \n  \"clinical_notes_summary\": \"A 2-3 sentence summary of the consultation\" \n} \n\nEnsure the JSON is valid and follows the specified schema."

```

```
{
  "_id": "suggestive_mode_v1",
  "prompt_name": "Suggestive Mode -- Clinical Safety Net",
  "version": 1,
  "model_target": "llama3.1-8b OR mistral-7b-v0.3",
  "system_prompt": "You are a clinical safety reviewer embedded in an outpatient clinic system. You will receive a doctor-patient consultation transcript AND the AI-generated clinical notes/prescription draft. Your job is to review both and flag potential issues the doctor should consider before approving.\n\nINPUT YOU WILL RECEIVE:\n- Patient metadata (name, age, gender, known allergies, medical history)\n- The full consultation transcript\n- The generated clinical notes and prescription JS\n\nOUTPUT REQUIREMENTS:\nReturn a single valid JSON object with exactly these keys:\n{\n  \"suggestions\": [\n    {\n      \"type\": \"OMISSION | CONTRAINDICATION | DOSAGE_CHECK | STANDARD_OF_CARE | INTERACTION_WARNING | FOLLOW_UP\", \n      \"severity\": \"LOW | MEDIUM | HIGH | CRITICAL\", \n      \"title\": \"Short one-line title of the flag\", \n      \"detail\": \"Detailed explanation of the concern\", \n      \"recommendation\": \"What the doctor should consider doing\", \n      \"source_quote\": \"Relevant quote from the transcript that triggered this flag, or 'N/A' if based on clinical knowledge\", \n      \"overall_risk_level\": \"GREEN | YELLOW | RED\", \n      \"summary\": \"One-paragraph summary of all flags and overall assessment\"\n    }\n  ]\n}\n\nFLAG TYPES:\n- OMISSION: Something mentioned in the transcript was not captured in the notes (e.g., patient mentioned dizziness but it is absent from the notes)\n- CONTRAINDICATION: A prescribed medication conflicts with the patient's known allergies or conditions\n- DOSAGE_CHECK: A dosage seems unusually high, low, or atypical for the diagnosis\n- STANDARD_OF_CARE: A commonly expected test, referral, or treatment for the diagnosis is missing\n- INTERACTION_WARNING: Two or more prescribed medications may interact\n- FOLLOW_UP: The consultation suggests follow-up is needed but none was scheduled, or the timeline seems inappropriate\n\nSEVERITY LEVELS:\n- LOW: Minor suggestion, unlikely to affect patient safety\n- MEDIUM: Worth reviewing, could improve care quality\n- HIGH: Potential patient safety issue, should be addressed before approval\n- CRITICAL: Likely dangerous -- e.g., allergy conflict, contraindicated drug, extreme dosage\n\nRISK LEVELS:\n- GREEN: No flags, or only LOW severity flags\n- YELLOW: At least one MEDIUM or HIGH flag\n- RED: At least one CRITICAL flag\n\nRULES:\n1. Only flag genuine clinical concerns. Do NOT generate flags just to fill the list.\n2. If there are no issues, return an empty suggestions array with GREEN risk level.\n3. Always quote the transcript when the flag is based on something the patient or doctor said.\n4. Be specific in recommendations -- say what test, what alternative drug, what dosage range.\n5. Do NOT override the doctor's clinical judgment. Frame all flags as suggestions, not commands.\n6. Do NOT include any text outside the JSON object. No preamble, no commentary.\n7. Cross-check the patient's allergy list against every prescribed medication.\n8. Cross-check the patient's medical history against every prescribed medication for contraindications.,\n\nuser_prompt_template": "PATIENT INFORMATION:\n- Name: {{patient_name}}\n- Age: {{patient_age}}\n- Gender: {{patient_gender}}\n- Known Allergies: {{patient_allergies}}\n- Medical History: {{patient_medical_history}}\n\nCONSULTATION TRANSCRIPT:\n{{transcript_text}}\n\nGENERATED CLINICAL NOTES & PRESCRIPTION:\n{{generated_clinical_notes_json}}\n\nReview the above transcript and generated notes. Flag any concerns as specified.,\n\n\"temperature\": 0.3,\n\"max_tokens\": 1500,\n\"created_at\": \"2026-02-24T00:00:00Z\", \n\"updated_at\": \"2026-02-24T00:00:00Z\"
}
```

Collection 3: generated_documents

Staging area for LLM-generated outputs. Doctor reviews here; only approved data moves to SQL prescriptions.

Key fields: consultation_id, generated_output, suggestive_output, doctor_edits[], status, approved_at

```
{
  "_id": "<auto>",
  "consultation_id": 42,
  "doctor_id": 7,
  "patient_id": 15,
  "document_type": "clinical_notes_and_prescription",
  "generated_output": {
    "patient_info": { "name": "...", "age": "...", "gender": "..."},
    "chief_complaint": "...",
    "history_of_present_illness": "...",
    "past_medical_history": "...",
    "allergies": "...",
    "vitals": { "blood_pressure": "...", "heart_rate": "...", "temperature": "...", "respiratory_rate": "...", "spo2": "..."},
    "examination_findings": "...",
    "diagnosis": "...",
    "medications": [
      { "name": "...", "dosage": "...", "frequency": "...", "duration": "...", "route": "...", "special_instructions": "..."}
    ],
    "lab_tests_ordered": [...],
    "follow_up": "...",
    "patient_instructions": "...",
    "clinical_notes_summary": "..."
  },
  "suggestive_output": {
    "suggestions": [
      { "type": "...", "severity": "...", "title": "...", "detail": "...", "recommendation": "...", "source_quote": "..."}
    ],
    "overall_risk_level": "GREEN | YELLOW | RED",
    "summary": "..."
  },
  "doctor_edits": [
    { "field_path": "medications[0].dosage", "old_value": "1000mg", "new_value": "500mg", "edited_at": 2026-02-24T10:35:00Z }
  ],
  "status": "pending_review | approved | rejected | revised",
  "approved_at": null,
  "created_at": "2026-02-24T10:30:00Z",
  "updated_at": "2026-02-24T10:30:00Z"
}
```

Collection 4: consultation_documents

Stores all large unstructured text per consultation: raw transcript, AI notes, AI suggestions, and doctor's final edits.

Key fields: consultation_id, transcript.full_text, ai_clinical_notes, ai_suggestions, doctor_edited_notes, edit_history[]

```
{
  "_id": "<auto>",
  "consultation_id": 42,
  "transcript": {
```

```

"file_path": "/data/transcripts/consult_42.txt",
"full_text": "<entire raw transcript from Faster-Whisper>"
},
"ai_clinical_notes": {
  "patient_info": {},
  "chief_complaint": "...",
  "diagnosis": "...",
  "medications": []
},
"ai_suggestions": {
  "suggestions": [],
  "overall_risk_level": "...",
  "summary": "..."
},
"doctor_edited_notes": null,
"edit_history": [
  { "field_path": "ai_clinical_notes.medications[0].dosage", "old_value": "1000mg", "new_value": "500mg", "edited_at": 20
    26-02-24T10:35:00Z }
],
"created_at": "2026-02-24T10:30:00Z",
"updated_at": "2026-02-24T10:30:00Z"
}

```

Part 3: SQL <-> NoSQL Relationship Map

SQL Table	NoSQL Collection	Link Key	Purpose
consultations	consultation_documents	consultation_id	Transcript + AI notes + doctor edits
consultations	generated_documents	consultation_id	LLM staging area before approval
prescriptions	generated_documents	consultation_id	Approved data flows NoSQL -> SQL
--	email_templates	(standalone)	Email format config
--	llm_prompts	(standalone)	LLM prompt config

Part 4: AI Prompt Definitions

The system uses two LLM calls per consultation, both stored in the llm_prompts MongoDB collection.

Prompt 1 -- Prescription & Clinical Notes Generator (prescription_generation_v1)

Purpose: Convert raw transcript into structured clinical notes + prescription JSON.

System prompt summary:

- Receives patient metadata + full transcript
- Must output a single JSON object with: patient_info, chief_complaint, history_of_present_illness, past_medical_history, allergies, vitals, examination_findings, diagnosis, medications[], lab_tests_ordered[], follow_up, patient_instructions, clinical_notes_summary
- Extracts ONLY what was explicitly said -- no invention
- Missing fields set to "Not mentioned in consultation"
- No text outside the JSON; dosages must include units

Config: temperature: 0.2 | max_tokens: 2048

Prompt 2 -- Suggestive Mode / Clinical Safety Net (suggestive_mode_v1)

Purpose: Review transcript + generated notes and flag potential clinical issues before doctor approval.

System prompt summary:

- Receives patient metadata + transcript + generated notes JSON
- Must output a JSON with suggestions[], overall_risk_level, summary
- Each suggestion has: type, severity, title, detail, recommendation, source_quote
- Flag types: OMISSION, CONTRAINDICATION, DOSAGE_CHECK, STANDARD_OF_CARE, INTERACTION_WARNING, FOLLOW_UP
- Severity: LOW to CRITICAL; Risk level: GREEN / YELLOW / RED
- Cross-checks allergies and medical history against all prescribed medications
- Frames flags as suggestions, never overrides doctor judgment
- Empty suggestions array + GREEN if no issues found

Config: temperature: 0.3 | max_tokens: 1500

Full prompt text for both is stored in the llm_prompts collection documents above (Collection 2, Documents 2A and 2B).

Part 5: Data Flow

Step	Action	SQL	NoSQL
1	Doctor starts consultation	consultations: new row (status=recording)	--
2	Audio transcribed (Faster-Whisper)	consultations: status to transcribing	consultation_documents: transcript saved
3	LLM generates notes + prescription	consultations: status to processing	generated_documents: AI draft saved; consultation_documents: ai_clinical_notes set
4	LLM runs safety review (suggestive mode)	--	generated_documents: suggestive_output saved; consultation_documents: ai_suggestions
5	Doctor reviews & edits draft	consultations: status to review	generated_documents: doctor_edits tracked; consultation_documents: doctor_edited_notes +
6	Doctor approves (NoSQL to SQL transfer)	prescriptions: NEW ROW (is_approved=TRUE);	generated_documents: status to approved
7	PDF generated (ReportLab)	prescriptions: pdf_path set	--
8	Email sent to patient	prescriptions: is_emailed=TRUE, emailed_at set; audit_logs: new entry	email_templates: READ for format

Key insight: NoSQL handles the messy beginning (transcripts, AI drafts, edits). SQL stores the finalized, relational end result (approved prescriptions, audit trail).